

We Built 15 Tools. The Agent Wanted a REPL.

Nuno Campos

Co-Founder & CTO, Witan Labs
Previously LangChain, LangGraph

AI Engineering London • April 2026

50% → 92%

4 months, 4 codebases, 29,000 lines of eval infrastructure

The single biggest decision: replacing 15 discrete tools
with one REPL

Spreadsheets are deceptively hard for AI

The image shows a screenshot of a complex Excel spreadsheet. The spreadsheet is organized into several tabs at the top, including 'Assumptions', 'Revenue', 'P&L', 'Balance Sheet', 'Cash Flow', 'Sensitivity', and 'Summary'. The main area is filled with a dense grid of data and formulas. The 'Revenue' tab is currently selected, showing a table with columns for 'Year' (2023 to 2030) and rows for various revenue categories. The 'P&L' tab shows a summary of revenue, cost of goods sold, and operating income. The 'Balance Sheet' tab shows assets, liabilities, and equity. The 'Cash Flow' tab shows operating, investing, and financing activities. The 'Sensitivity' tab shows a sensitivity analysis for key assumptions. The 'Summary' tab provides a high-level overview of the financial model. The spreadsheet is color-coded, with yellow highlighting used for certain sections and formulas. The overall structure is complex and difficult to navigate, especially for an AI model.

A human sees:

Revenue table, assumptions,
P&L summary, color-coded
sections, structure

An LLM sees:

10,000 cell values, cryptic
formulas, formatting metadata
it has no intuition for

```
=SUMPRODUCT((B$3:B$50="NEW")*(G$3:G$50))
```

Attempt 1: Turn the spreadsheet into a database

Excel → SQLite

Every cell, formula, merged region, color block stored in relational tables

Agent: **LangGraph + GPT-5**

Tools: SQL query, write cell, insert row, delete range

Each tool call = a separate LLM round-trip

Result:
50%

50% → 73%

Fixed one character in the extraction pipeline.

The agent had looked confused. Our instinct was to blame the model.

The actual problem: the agent was reasoning correctly over corrupted inputs.

Attempt 2: Three agents, not one

Block Discovery

Identifies workbook structure

Low reasoning effort

Edit Agent

Modifies workbooks via 5-step process

High reasoning effort

Question Agent

Answers questions about contents

Read-only

Key finding: "Define the end state before you touch a cell" was the single highest-leverage prompt instruction. The 5-step process (disambiguate, define end state, plan, execute, verify) caught errors during planning, not execution.

The dead ends

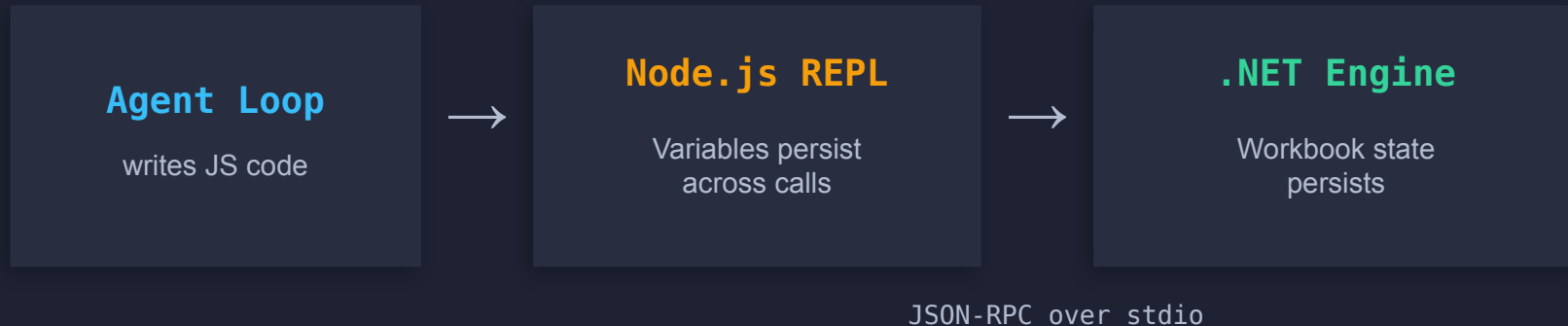
We spent two weeks searching for the right static representation.

Representation	Why it failed
TSV views	Lost formatting and structural context
SQL views	Flat tables didn't capture visual structure
HTML tables	Too verbose, consumed too many tokens
DOT graphs	Useful for analysis, not for interaction
XML / XSLT	LLM struggled with the syntax

The answer was to stop looking for a static representation entirely.

November 23rd: The REPL

One tool. One persistent Node.js REPL. A rich spreadsheet API.



30+ functions: `listSheets`, `findCells`, `readRange`, `traceToInputs`, `setCells`, `evaluateFormula` ...

Sandboxed: no filesystem, no network, no subprocesses

Before vs. After

BEFORE

10-15 tool calls

```
Tool call 1: list_sheets()
  → ["Summary", "Data", "Inputs"]
Tool call 2: read_range("Summary!A1:D10")
  → cell values
Tool call 3: find_cells("Revenue")
  → [matches]
Tool call 4: read_cell("Summary!C10")
  → "$1,234,567"
...
```

AFTER

1 tool call

```
const sheets = await xlsx.listSheets(wb);

const summary = await xlsx.readRangeTsv(
  wb, `${sheets[0].name}!A1:D10`
);

const revenue = await xlsx.findCells(
  wb, "Revenue", { context: 1 }
);

console.log(sheets, summary, revenue);
```

One call, three operations, all results visible together.

But it goes further than just batching.

Why the REPL actually worked

1

Composability

Multiple operations per call,
results feed into each other

2

State persistence

Variables survive across calls,
build understanding incrementally

3

Flexible exploration

Conditionals, loops, error recovery
— in a single tool call

4

Trivial API evolution

Add a function, agent uses it.
No tool-layer plumbing.

```
let results = await xlsx.findCells(wb, "Net Revenue");
if (!results.length)
  results = await xlsx.findCells(
    wb, ["Total Revenue", "Net Sales", "Revenue"]
  );
```

Three hard rules

Each one a scar from a previous failure. Embedded in the system prompt.

1 "Zero new formula errors"

After every write, check for errors.
If your edit broke a formula, fix it before moving on.

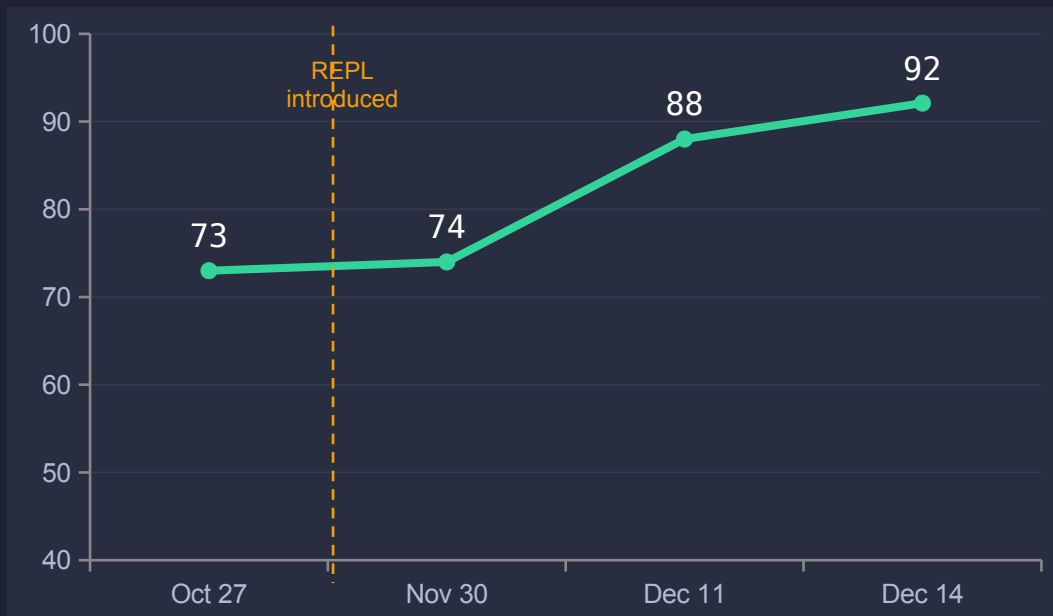
2 "Use the model, not JavaScript math"

Read calculated values from the formula engine.
Never substitute your own arithmetic.

3 "Always cite sheet + address"

Every answer must reference where the data came from.
Summary!C10, not just "\$1.2M".

74% to 92% in two weeks



92.1%

152/165 tasks

0

timeouts

50s

avg runtime

18 points in two weeks from compounding small gains:
better search, new API functions, improved docs, backend bug fixes

The test that contradicted our thesis

exec / REPL

The 92% approach
Already proven internally

(not part of this test)

verify CLI

Separate commands:
find, calc, render, lint

vs. openpyxl on 20 QnA tasks

Same model (Claude Opus 4.6) • Same runner • Same 20 tasks

We expected: our sophisticated tool wins.

openpyxl won.

	openpyxl	Witan CLI
Pass rate	85%	70%
Avg tool calls	21	42

The simpler approach won by 15 points.

Why it lost (the mundane reasons)

1 A performance bug

Each CLI command took 50-130s. Per-cell recalculation query instead of batch. 20+ ops per task → timeout.

2 Shell escaping

Sheet names with spaces ("SW Selections") broke across three quoting layers: Claude Code SDK → zsh → .NET

3 Documentation was backwards

SKILL.md had the wrong quoting examples.
The agent followed our wrong instructions faithfully.

Meanwhile, openpyxl: **Python strings. Millisecond file access. Zero infrastructure. Zero quoting issues.**

What the failure revealed

Two products emerged from one failed test:

exec

The REPL externalized as a CLI command any agent can use. Single invocation, full exploration.

verify

render, calc, lint as a lightweight add-on for agents that already have their own spreadsheet tools.

The most important test isn't the one that confirms your assumptions — it's the one that reveals your product.

Domain knowledge outlived every tool

openpyxl



xlwings



Witan CLI



REPL



Financial domain knowledge

Improved results on all four backends

- How to interpret margins, profitability,
- revenue cascades through COGS
- Model type recognition
- (DCF, LBO, three-statement)
- Communication conventions
- (\$1.2M not \$1,234,567.89)
- *"Never calculate in your head what the spreadsheet can calculate for you"*

If you're building a domain-specific agent, the domain knowledge is the product.
The tools are replaceable.

Stop using LLMs to grade LLMs

Started with: LLM-as-judge comparing output to gold standard

Problem: Inconsistent judgments that masked real regressions

Strategy	Method
Content	Set similarity (Jaccard $\geq 70\%$)
Structure	Row-level sequence alignment (dynamic programming)
Visual	Font / color / format comparison ($\geq 90\%$)
Scenarios	Set inputs, recalculate, compare outputs
Text	LLM grading — only for genuinely subjective answers

29,000 lines of eval code **568** commits

Infrastructure bugs masquerade as reasoning failures

Agent can't find data

50% → 73%

→ **One-character extraction bug**

Agent uses wrong quoting

Agent trusted docs

→ **SKILL.md had backwards examples**

Agent times out constantly

50-130s per op

→ **Per-cell recalc query, not batch**

Agent retries endlessly

11-40% success

→ **API returning empty results**

When the agent seems confused, suspect the plumbing.

What generalizes

- 1 If your agent makes many small sequential tool calls that compose into larger operations, you've reinvented a bad scripting language. Give it a real one.
- 2 Structured reasoning > better tools. "Define the end state before you act" caught more errors than any tool improvement.
- 3 Domain knowledge is the product. Tools are replaceable. What the agent knows about your domain is not.
- 4 Evaluate deterministically. LLM-as-judge is fine for vibes. For measuring progress, use programmatic comparison.
- 5 Blame the plumbing first. Agent "confusion" is usually a bug in the data, the docs, or the infrastructure.

We kept trying to constrain the agent
into tighter interactions.

It kept wanting to program.

Once we let it, everything changed.

Research log: github.com/witanlabs/research-log

CLI & Skills: github.com/witanlabs/witan-cli

Nuno Campos

Witan Labs